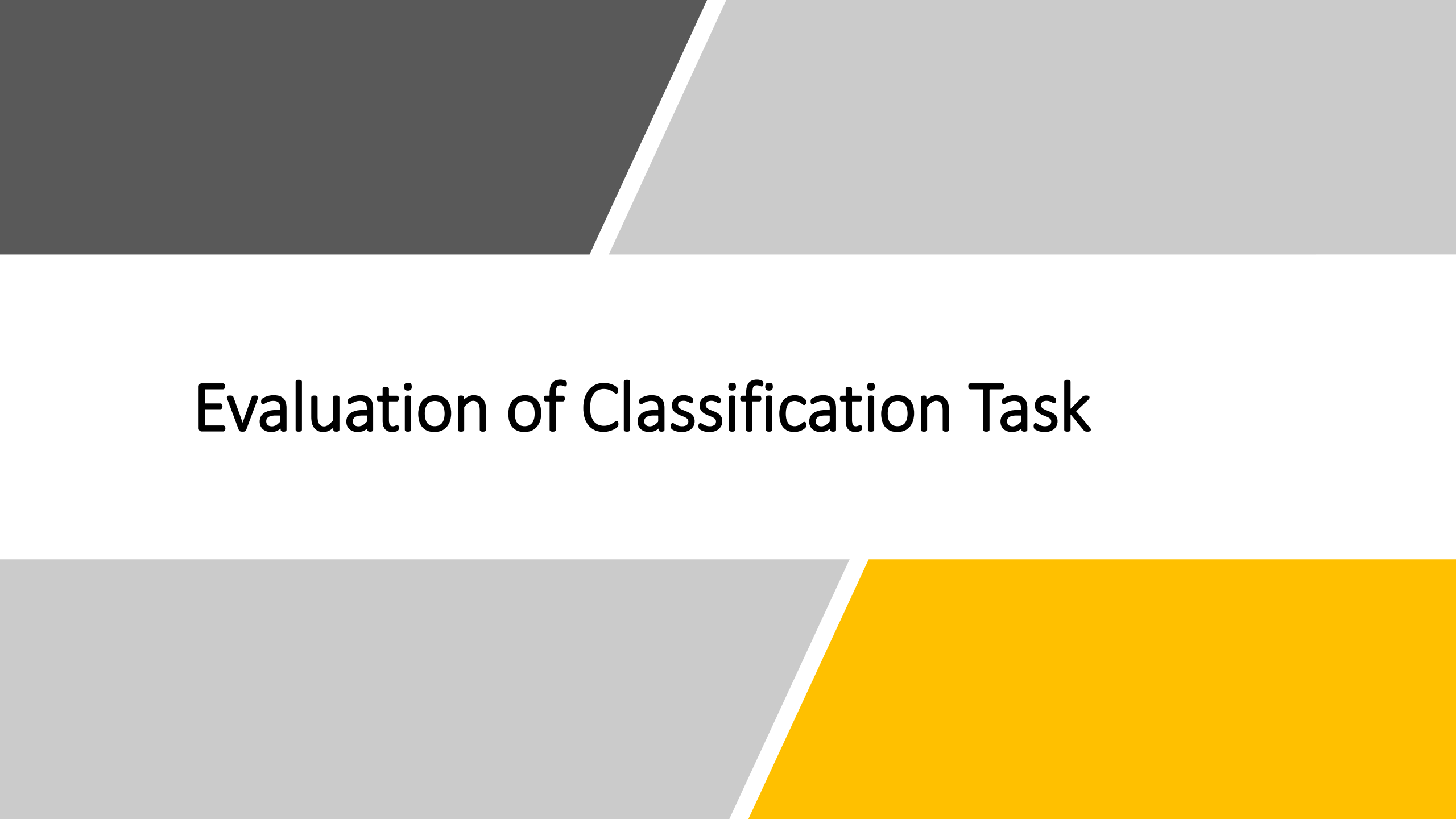




Evaluation of Classification Task

Confusion Matrix

- A **confusion matrix** is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known.
- The **confusion matrix** itself is relatively simple to understand, but the related terminology can be **confusing**.

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

Confusion Matrix

- **true positives (TP):** These are cases in which we predicted yes (they have the disease), and they do have the disease.
- **true negatives (TN):** We predicted no, and they don't have the disease.
- **false positives (FP):** We predicted yes, but they don't actually have the disease. (Also known as a "Type I error.")
- **false negatives (FN):** We predicted no, but they actually do have the disease. (Also known as a "Type II error.")

		n=165	
		Predicted: NO	Predicted: YES
Actual:	NO	TN = 50	FP = 10
	YES	FN = 5	TP = 100
		55	110

Performance Metrics computed from Confusion Matrix

- Accuracy
- Precision
- Recall
- F-Measure

Performance Metrics computed from Confusion Matrix

n=165		Predicted: NO	Predicted: YES	
Actual: NO		TN = 50	FP = 10	60
Actual: YES		FN = 5	TP = 100	105
		55	110	

- **Accuracy:** Overall, how often is the classifier correct?
 - $(TP+TN)/total = (100+50)/165 = 0.91$
- **Misclassification Rate:** Overall, how often is it wrong?
 - $(FP+FN)/total = (10+5)/165 = 0.09$

Performance Metrics computed from Confusion Matrix

Precision - Precision is the ratio of correctly predicted positive observations to the total predicted positive observations. The question that this metric answer is of all passengers that labeled as survived, how many actually survived? High precision relates to the low false positive rate. We have got 0.788 precision which is pretty good.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall (Sensitivity) - Recall is the ratio of correctly predicted positive observations to the all observations in actual class - yes. The question recall answers is: Of all the passengers that truly survived, how many did we label? We have got recall of 0.631 which is good for this model as it's above 0.5.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Performance Metrics computed from Confusion Matrix

F1 score - F1 Score is the weighted average of Precision and Recall. Therefore, this score takes both false positives and false negatives into account. Intuitively it is not as easy to understand as accuracy, but F1 is usually more useful than accuracy, especially if you have an uneven class distribution. Accuracy works best if false positives and false negatives have similar cost. If the cost of false positives and false negatives are very different, it's better to look at both Precision and Recall. In our case, F1 score is 0.701.

$$\text{F1 Score} = 2 * (\text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision})$$



K-Means Clustering



- Suppose you are managing a supermarket.
- You have data about how much money different customers spend and how frequently they visit.
- How can you group customers with similar shopping patterns?

K-Means Clustering

- it finds groups (clusters) of similar items in your data automatically, without knowing the labels beforehand

What Is K-Means?

It is an **unsupervised learning algorithm** used for **clustering**.

- “Unsupervised” means there are **no predefined labels** (no Pass/Fail, Cat/Dog).
- The algorithm looks at your data and tries to **find patterns** or **groups** on its own.
- “K-Means groups similar data points together based on their closeness (distance).”

Student	Math Marks	English Marks
A	90	85
B	88	80
C	30	40
D	25	35
E	70	65

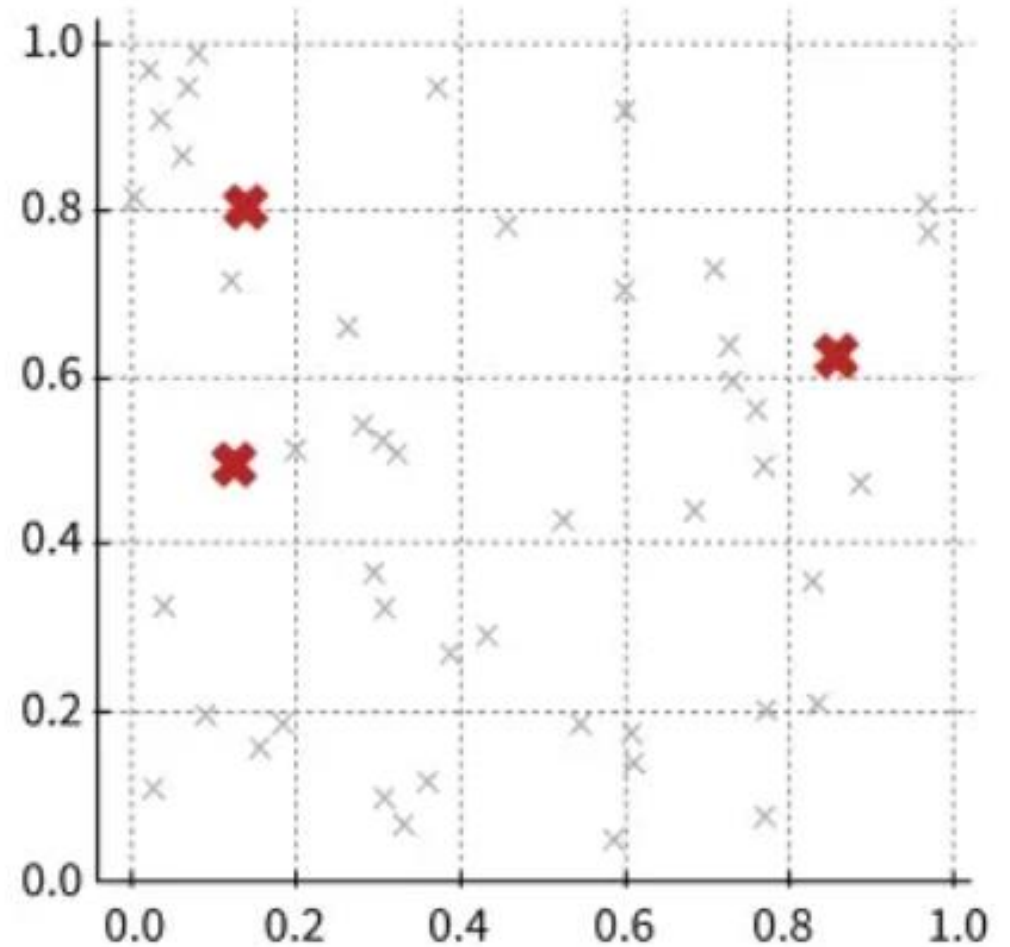
How K-Means Works (Step-by-Step)

Step	Explanation
1. Choose K	Decide how many clusters (groups) you want (e.g., $K = 3$).
2. Pick Initial Centers (Centroids)	The algorithm picks K random points as starting “centers.”
3. Assign Points	Each data point is assigned to the nearest centroid.
4. Recalculate Centroids	Compute new cluster centers by averaging the points in each cluster.
5. Repeat	Keep repeating steps 3–4 until the centroids don’t move much (convergence).

Choose Initial Centroids

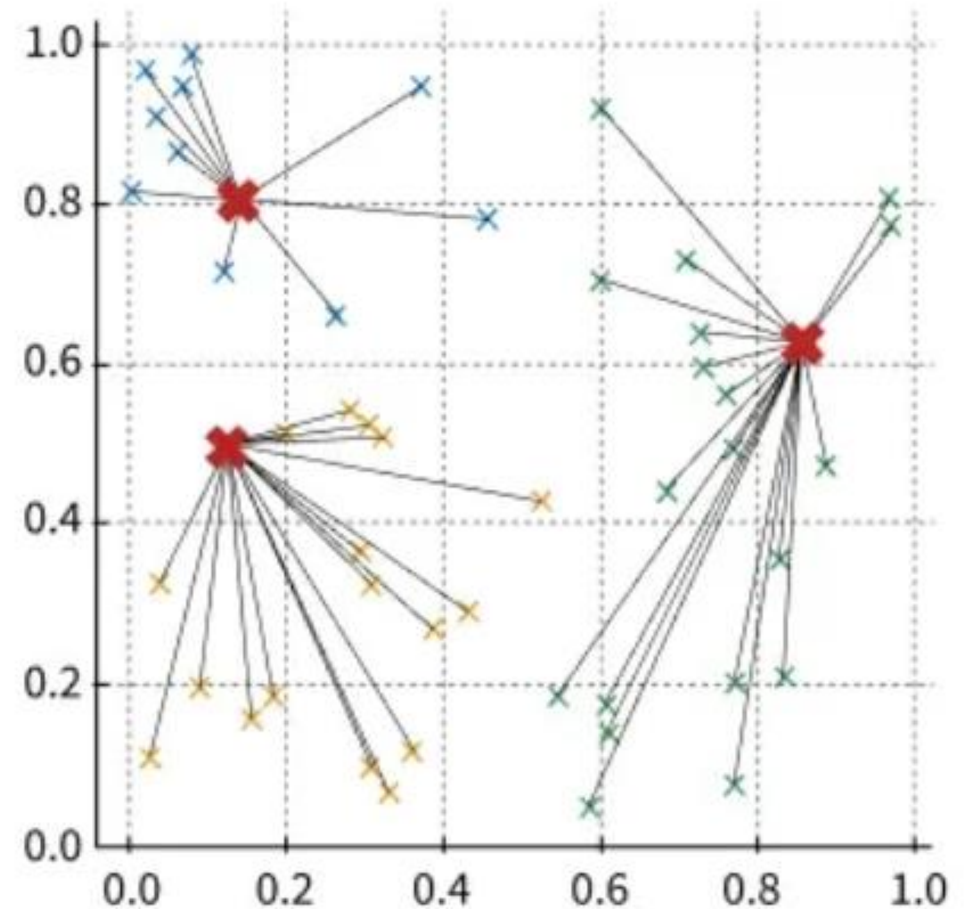
Centroids are randomly chosen from the data points. These represent the initial cluster centers.

✖ Centroids ✕ Data Points



Each point is assigned to the nearest centroid, forming clusters

- ✖ Centroids
- ✖ Cluster 1
- ✖ Cluster 2
- ✖ Cluster 3



Update Centroids

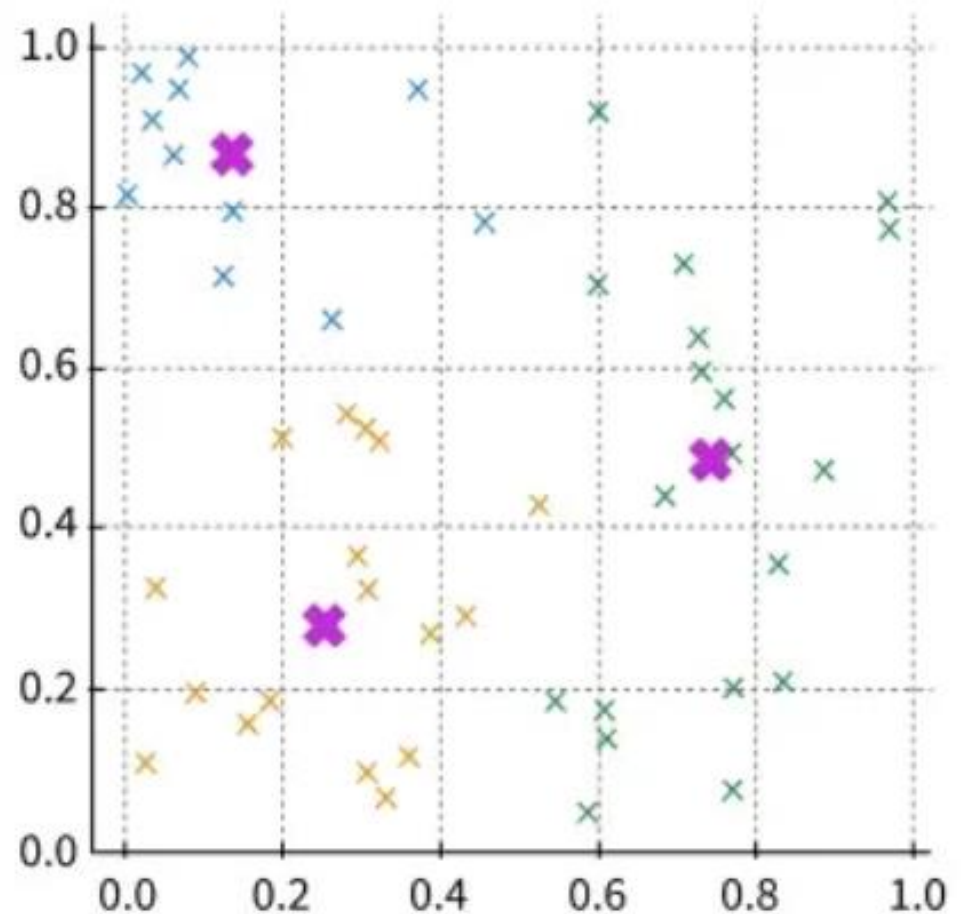
Centroids are recalculated as the mean of the points in each cluster

✱ New Centroids

✕ Cluster 1

✕ Cluster 2

✕ Cluster 3



Repeat Until Convergence

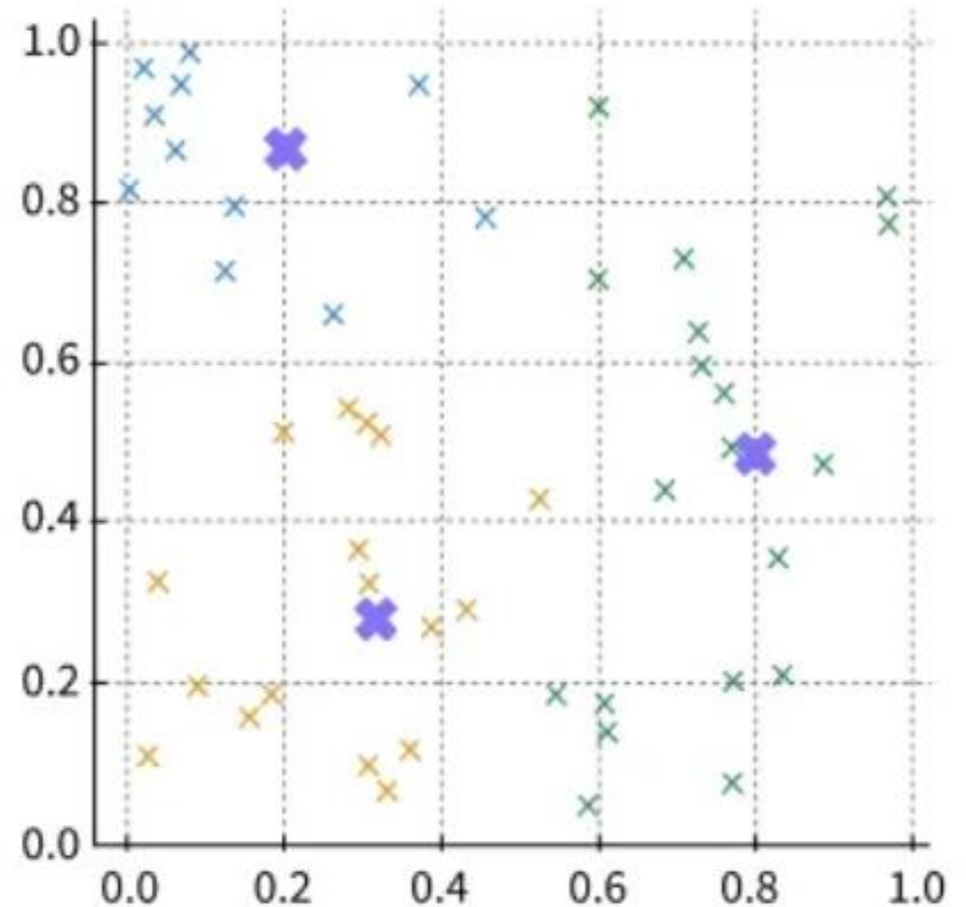
This process repeats until the centroids stabilize and do not move further.

✖ Final Centroids

✖ Cluster 1

✖ Cluster 2

✖ Cluster 3





K-Means Clustering Example

Step 1: Import libraries

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import pandas as pd
```

Step 2: Create sample data (Math vs English marks)

```
data = {
    "Math": [90, 88, 30, 25, 70, 60, 32, 95, 72, 28],
    "English": [85, 80, 40, 35, 65, 60, 42, 90, 68, 38]
}
```

```
df = pd.DataFrame(data)
```

Step 3: Create K-Means model (choose K=3)

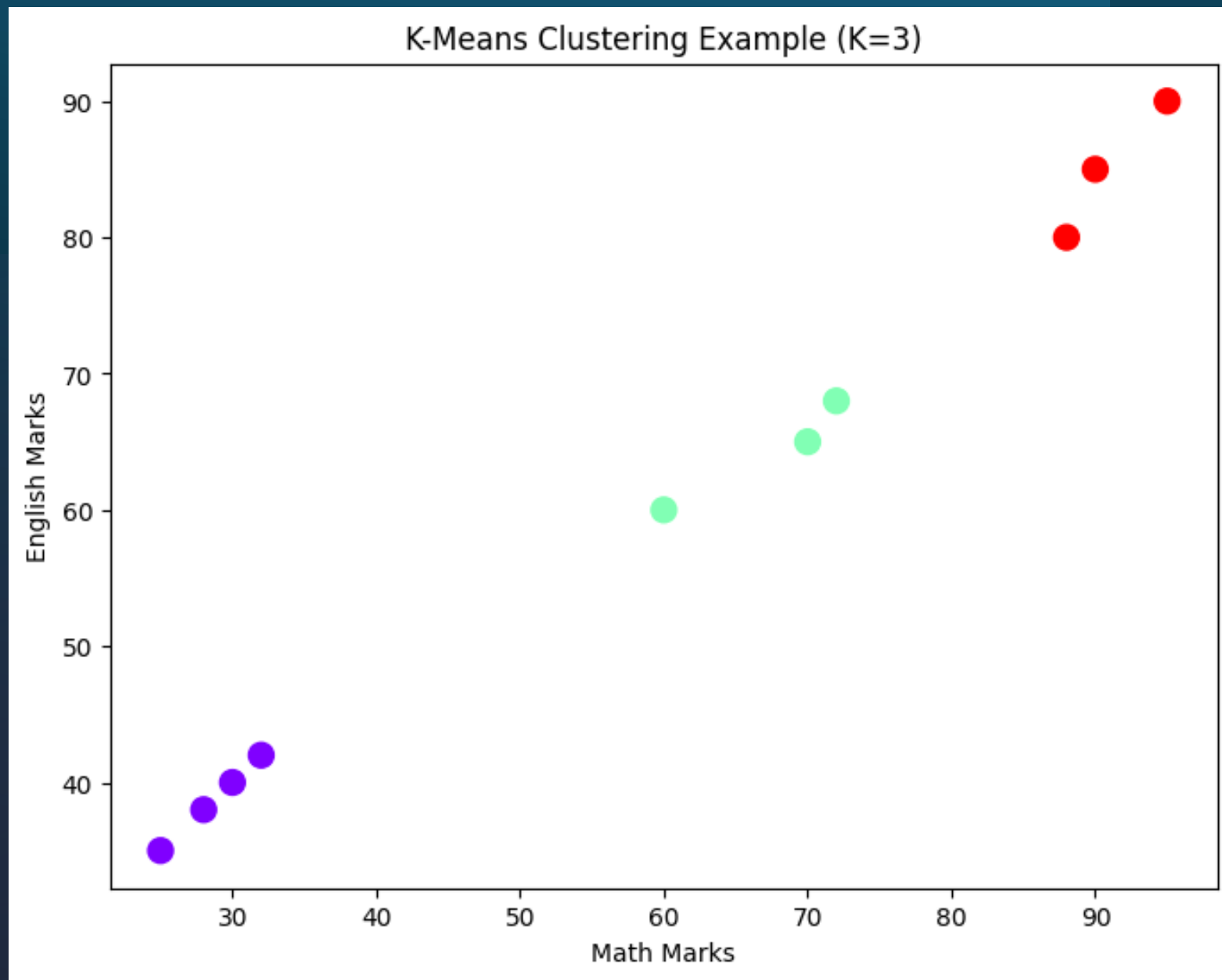
```
model = KMeans(n_clusters=3, random_state=42)
model.fit(df)
```

Step 4: Get cluster labels and add to DataFrame

```
df["Cluster"] = model.labels_
```

Step 5: Visualize the clusters

```
plt.figure(figsize=(8,6))
plt.scatter(df["Math"], df["English"], c=df["Cluster"], cmap="rainbow", s=100)
plt.xlabel("Math Marks")
plt.ylabel("English Marks")
plt.title("K-Means Clustering Example (K=3)")
plt.show()
```



Choosing K (Number of Clusters)

- You can try different K values (like 2, 3, 4, etc.) and see which fits best.
- To find the best K, use the **Elbow Method** (optional for first-year):
 - Plot the total distance (inertia) vs number of clusters.
 - The point where the graph bends (“elbow”) gives the optimal K.

Comparison with Previous Algorithm (DT)

Algorithm	Type	Goal	Example
Decision Tree	Supervised	Predict class labels	Pass/Fail
K-Means	Unsupervised	Find hidden groups	Customer Segmentation

Summary

Concept

Meaning

K-Means

Clustering algorithm that groups similar data points

K

Number of clusters you want

Centroid

The center point of each cluster

Goal

Minimize the distance between points and their cluster center

Use Cases

Market segmentation, image compression, document grouping, customer analysis

- K-Means is like a sorting machine — it automatically groups similar things together without being told what they are. It's simple, powerful, and widely used in data science applications.

K-Means Clustering Algorithm – Solved Example

- Use K Means clustering to cluster the following data into two groups.



- Data Points: { 2, 4, 10, 12, 3, 20, 30, 11, 25 }
- The distance function used is Euclidean distance.
- Initial cluster centroid are M1 = 4 and M2 = 11.

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

M1: 4

M2: 11

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2				
4				
10				
12				
3				
20				
30				
11				
25				

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

M1: 4

M2: 11

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2			
4	0			
10	6			
12	8			
3	1			
20	16			
30	26			
11	7			
25	21			

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

M1: 4

M2: 11



Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2	9		
4	0	7		
10	6	1		
12	8	1		
3	1	8		
20	16	9		
30	26	19		
11	7	0		
25	21	14		

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

M1: 4

M2: 11

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2	9	C1	
4	0	7	C1	
10	6	1	C2	
12	8	1	C2	
3	1	8	C1	
20	16	9	C2	
30	26	19	C2	
11	7	0	C2	
25	21	14	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

M1: 4

M2: 11

Therefore

C1= {2, 4, 3}

C2= {10, 12, 20, 30, 11, 25}

New Centroids: ✓

M1: 3 ✓

M2: 18 ✓

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2	9	C1	
4	0	7	C1	
10	6	1	C2	
12	8	1	C2	
3	1	8	C1	
20	16	9	C2	
30	26	19	C2	
11	7	0	C2	
25	21	14	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:

M1: 3

M2: 18

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	1	16	C1	C1 ✓
4	1	14	C1	C1
10	7	8	C2	C1
12	9	6	C2	C2
3	0	15	C1	C1
20	17	2	C2	C2
30	27	12	C2	C2
11	8	7	C2	C2
25	22	7	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:

M1: 3

M2: 18

Therefore

C1= {2, 4, 10, 3}

C2= {12, 20, 30, 11, 25}

New Centroids:

M1: 4.75

M2: 19.6

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	1	16	C1	C1
4	1	14	C1	C1
10	7	8	C2	C1
12	9	6	C2	C2
3	0	15	C1	C1
20	17	2	C2	C2
30	27	12	C2	C2
11	8	7	C2	C2
25	22	7	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:

M1: 4.75

M2: 19.6

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2.75	17.6	C1	
4	0.75	15.6	C1	
10	5.25	9.6	C1	
12	7.25	7.6	C2	
3	1.75	16.6	C1	
20	15.25	0.4	C2	
30	25.25	10.4	C2	
11	6.25	8.6	C2	
25	20.25	5.4	C2	

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:

M1: 4.75

M2: 19.6

Therefore

C1= {2, 4, 10, 11, 12, 3}

C2= {20, 30, 25}

New Centroids:

M1: 7 ✓

M2: 25 ✓

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2.75	17.6	C1	C1
4	0.75	15.6	C1	C1
10	5.25	9.6	C1	C1
12	7.25	7.6	C2	C1
3	1.75	16.6	C1	C1
20	15.25	0.4	C2	C2
30	25.25	10.4	C2	C2
11	6.25	8.6	C2	C1
25	20.25	5.4	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:

M1: 7

M2: 25

Final Cluster are:

C1= {2, 4, 10, 11, 12, 3} ✓

C2= {20, 30, 25} ✓

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	5	23	C1	C1
4	3	21	C1	C1
10	3	15	C1	C1
12	5	13	C1	C1
3	4	22	C1	C1
20	13	5	C2	C2
30	23	5	C2	C2
11	4	14	C1	C1
25	18	0	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

- Apply K Means clustering algorithm to cluster the following data points into two groups.
- Data Points:
 - (185, 72), (170, 56), (168, 60), (179, 68), (182, 72), (188, 77)
- The distance function used is **Euclidean distance**.
- First two data points as initial centroids.

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

C1: (185, 72)

C2: (170, 56)

Data Points	Distance to		Cluster	New Cluster
	(185, 72)	(170, 56)		
(185, 72)				
(170, 56)				
(168, 60)				
(179, 68)				
(182, 72)				
(188, 77)				

$$d\{(x_2, x_1), (y_2, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

C1: (185, 72)

C2: (170, 56)

Data Points	Distance to		Cluster	New Cluster
	(185, 72)	(<u>170, 56</u>)		
(185, 72)	0			
(170, 56)	21.93			
(168, 60)	20.81			
(179, 68)	7.21			
(182, 72)	3			
(188, 77)	5.83			

$$d\{(x_2, y_2), (x_1, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

C1: (185, 72)

C2: (170, 56)

Data Points	Distance to		Cluster	New Cluster
	(185, 72)	(170, 56)		
(185, 72)	0	21.93		
(170, 56)	21.93	0		
(168, 60)	20.81	4.47		
(179, 68)	7.21	15		
(182, 72)	3	20		
(188, 77)	5.83	27.66		

$$d\{(x_2, x_1), (y_2, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

C1: (185, 72)

C2: (170, 56)

Data Points	Distance to		Cluster	New Cluster
	(185, 72)	(170, 56)		
(185, 72)	0	21.93	C1	
(170, 56)	21.93	0	C2	
(168, 60)	20.81	4.47	C2	
(179, 68)	7.21	15	C1	
(182, 72)	3	20	C1	
(188, 77)	5.83	27.66	C1	

$$d\{(x_2, x_1), (y_2, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Initial Centroids:

C1: (185, 72)

C2: (170, 56)

New Centroids:

C1: (183.5, 72.25)

C2: (169, 58)

Data Points	Distance to		Cluster	New Cluster
	(185, 72)	(170, 56)		
(185, 72)	0	21.93	C1	
(170, 56)	21.93	0	C2	
(168, 60)	20.81	4.47	C2	
(179, 68)	7.21	15	C1	
(182, 72)	3	20	C1	
(188, 77)	5.83	27.66	C1	

$$d\{(x_2, y_2), (x_1, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Centroids:

C1: (183.5, 72.25) —

C2: (169, 58)

Data Points	Distance to		Cluster	New Cluster
	(183.5, 72.25)	(169, 58)		
(185, 72)			C1	
(170, 56)			C2	
(168, 60)			C2	
(179, 68)			C1	
(182, 72)			C1	
(188, 77)			C1	

$$d\{(x_2, x_1), (y_2, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Centroids:

C1: (183.5, 72.25)

C2: (169, 58)

Data Points	Distance to		Cluster	New Cluster
	(183.5, 72.25)	(169, 58)		
(185, 72)	1.52	22.02	C1	
(170, 56)	21.13	2.83	C2	
(168, 60)	19.76	2.00	C2	
(179, 68)	6.19	14.87	C1	
(182, 72)	1.52	19.80	C1	
(188, 77)	6.54	27.59	C1	

$$d\{(x_2, x_1), (y_2, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Centroids:

C1: (183.5, 72.25)

C2: (169, 58)

Data Points	Distance to		Cluster	New Cluster
	(183.5, 72.25)	(169, 58)		
(185, 72)	1.52	22.02	C1	C1
(170, 56)	21.13	2.83	C2	C2
(168, 60)	19.76	2.00	C2	C2
(179, 68)	6.19	14.87	C1	C1
(182, 72)	1.52	19.80	C1	C1
(188, 77)	6.54	27.59	C1	C1

$$d\{(x_2, y_2), (x_1, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Centroids:

C1: (183.5, 72.25)

C2: (169, 58)

Final Centroids:

C1: (183.5, 72.25) ✓

C2: (169, 58) ✓

Data Points	Distance to		Cluster	New Cluster
	(183.5, 72.25)	(169, 58)		
(185, 72)	1.52	22.02	C1	C1
(170, 56)	21.13	2.83	C2	C2
(168, 60)	19.76	2.00	C2	C2
(179, 68)	6.19	14.87	C1	C1
(182, 72)	1.52	19.80	C1	C1
(188, 77)	6.54	27.59	C1	C1

$$d\{(x_2, y_2), (x_1, y_1)\} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Confusion Matrix



A **CONFUSION MATRIX** IS A TABLE THAT IS OFTEN USED TO DESCRIBE THE PERFORMANCE OF A CLASSIFICATION MODEL (OR "CLASSIFIER") ON A SET OF TEST DATA FOR WHICH THE TRUE VALUES ARE KNOWN.



THE **CONFUSION MATRIX** ITSELF IS RELATIVELY SIMPLE TO UNDERSTAND, BUT THE RELATED TERMINOLOGY CAN BE **CONFUSING**.

n=165	Predicted: NO	Predicted: YES
Actual: NO	50	10
Actual: YES	5	100

Confusion Matrix

- **true positives (TP):** These are cases in which we predicted yes (they have the disease), and they do have the disease.
- **true negatives (TN):** We predicted no, and they don't have the disease.
- **false positives (FP):** We predicted yes, but they don't actually have the disease. (Also known as a "Type I error.")
- **false negatives (FN):** We predicted no, but they actually do have the disease. (Also known as a "Type II error.")

n=165	Predicted: NO	Predicted: YES	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

Performance Metrics computed from Confusion Matrix

- Accuracy
- Precision
- Recall
- F-Measure

Performance Metrics computed from Confusion Matrix

n=165		Predicted: NO	Predicted: YES	
Actual: NO		TN = 50	FP = 10	60
Actual: YES		FN = 5	TP = 100	105
		55	110	

- **Accuracy:** Overall, how often is the classifier correct?
 - $(TP+TN)/total = (100+50)/165 = 0.91$
- **Misclassification Rate:** Overall, how often is it wrong?
 - $(FP+FN)/total = (10+5)/165 = 0.09$

Performance Metrics computed from Confusion Matrix

Precision - Precision is the ratio of correctly predicted positive observations to the total predicted positive observations. The question that this metric answer is of all passengers that labeled as survived, how many actually survived? High precision relates to the low false positive rate. We have got 0.788 precision which is pretty good.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall (Sensitivity) - Recall is the ratio of correctly predicted positive observations to the all observations in actual class - yes. The question recall answers is: Of all the passengers that truly survived, how many did we label? We have got recall of 0.631 which is good for this model as it's above 0.5.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Performance Metrics computed from Confusion Matrix

F1 score - F1 Score is the weighted average of Precision and Recall. Therefore, this score takes both false positives and false negatives into account. Intuitively it is not as easy to understand as accuracy, but F1 is usually more useful than accuracy, especially if you have an uneven class distribution. Accuracy works best if false positives and false negatives have similar cost. If the cost of false positives and false negatives are very different, it's better to look at both Precision and Recall. In our case, F1 score is 0.701.

$$\text{F1 Score} = 2 * (\text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision})$$